

Programs and Assignments

How `x = 2; print x;` becomes output

Gregory S. DeLozier, PhD

The New Program

```
x = 2;  
print x;
```

2

What must the interpreter remember?

Expressions and Statements

An **expression** produces a value.

```
x + 3 * 4
```

A **statement** performs an action.

```
x = 2  
print x
```

A Program Contains Statements

```
program ::= statement_list  
  
statement ::= assignment_statement  
           | print_statement
```

A statement list contains at least one statement.

Separating Statements

Source	Valid?
<code>x = 2</code>	Yes
<code>x = 2; print x</code>	Yes
<code>;;;x = 2;;;print x;;;</code>	Yes
<code>x = 2 print x</code>	No
empty input	No

Comments and Statement Separators

```
x = 2; // Keep the separator.  
print x
```

The newline does not replace the semicolon.

```
x = 2 // ; is part of the comment.  
print x
```

The second program has no separator between statements.

Statement List Grammar

```
statement_list ::= { ";" } statement  
                { ";" { ";" } statement }  
                { ";" }
```

Braces mean zero or more repetitions.

The Two Statement Forms

```
assignment_statement ::= <identifier> "=" expression
```

```
print_statement ::= "print" expression
```

```
x = 2  
print x
```

Identifiers in Expressions

```
factor ::= <number>  
        | <identifier>  
        | "(" expression ")"
```

```
y = x + 3 * 4
```

What value does x have?

Expressions Carry Forward

```
term ::= unary { ("*" | "/") unary }  
unary ::= "-" unary | factor
```

```
x = -1.5  
y = 3 * -2
```

The Program Tree

```
program
  statement_list
    assign
      target: identifier x
      expression: number 2
    print
      identifier x
```

The tree preserves statement order.

The Environment

An environment maps names to values.

Name	Value
x	2

The program begins with an empty environment.

Assignment Changes the Environment

```
x = 2
```

1. Evaluate the expression 2.
2. Associate the result with x.

```
{ } becomes { "x": 2 }
```

Print Reads the Environment

```
print x
```

1. Look up x.
2. Obtain 2.
3. Print 2.

An unknown name causes an evaluation error.

Parent Environments

```
parent = {"x": 10, "y": 20}  
environment = {"$PARENT": parent, "x": 3}
```

Search here, then outward. **The first binding wins.**

Lookup	Result
x	3 from the current environment
y	20 from the enclosing environment
z	Error: no binding in the chain

Assignment stays local: `y = 7` leaves the parent's `y` at 20.

One Environment for the Program

Statement	Environment afterward	Output
<code>x = 2</code>	<code>{"x": 2}</code>	
<code>print x</code>	<code>{"x": 2}</code>	2
<code>y = x + 3 * 4</code>	<code>{"x": 2, "y": 14}</code>	
<code>print y</code>	<code>{"x": 2, "y": 14}</code>	14

Assignment Is a Statement

Valid:

```
x = 2;  
print x;
```

Invalid in Topic 2:

```
print (x = 2)
```

Real Parser Code: Go

```
case token.ADD, token.SUB, token.NOT,  
    token.XOR, token.AND, token.TILDE:  
    pos, op := p.pos, p.tok  
    p.next()  
    x := p.parseUnaryExpr()  
    return &ast.UnaryExpr{OpPos: pos, Op: op, X: x}
```

Same move: consume the operator, parse the operand, build a tree node.

Topic 2

Vertex can now:

- execute a nonempty statement list
- assign values to names
- use names in arithmetic expressions
- print expression values

The next lecture follows these rules through the code.